

we burned 48 gpu hours training on 96 million tokens

alosh denny

superfloat · rootconf 2026

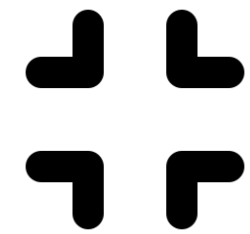
portrait photo goes here

this is a talk about failure. mostly mine.

but it starts somewhere else entirely.

i build a number format, **and a chip to run it on.** before anyone commits silicon, somebody has to answer one question.

how few bits does a model actually need?



quantisation

compressing a model so it uses less memory, less power, and generates tokens faster.

it is rounding. every number in the model, written with fewer digits. the model gets smaller and cheaper. the question is what it forgets.

and four people have to agree to do it.



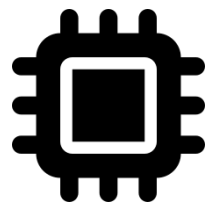
———— **cost.** which gpu tier you are willing to pay for.



———— **platform.** runpod, modal, or the cloud you already have.



———— **quality.** how far you compress before it stops being smart.



———— **hardware.** nvidia, or the faster new silicon with no community yet.

in most organisations you can settle two of those. maybe three. **never four.**

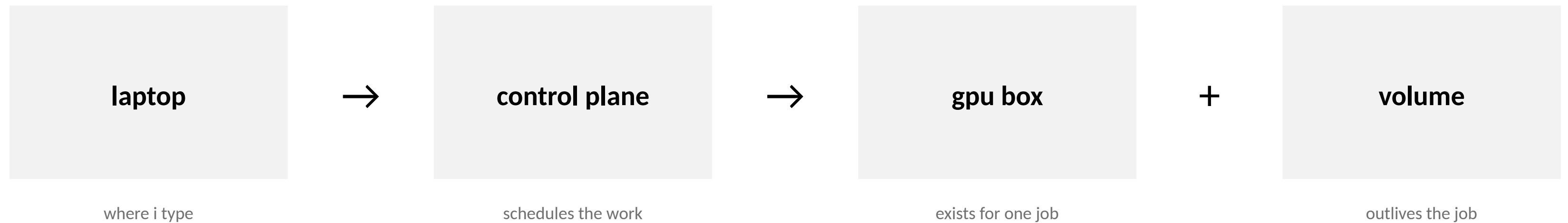
8 bits

the floor. compress past it and the model stops being intelligent

at six bits, a 4b model drops to **63**. at eight it is level with sixteen.

to find that out, you train a lot of models.

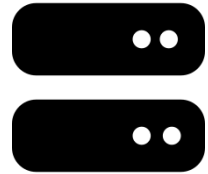
878 runs. four platforms in a year. none of them on hardware **i own.**



four things. **three of them are somebody else's.**



_____ **modal.** a python function that runs on an h100. pay by the second. the container dies at 24 hours, always.



_____ **runpod.** a machine you rent by the hour. yours until you stop paying. keeps nothing unless you attach a disk.



_____ **before those,** a gpu under my desk and a shared lab machine.

llama 3.2, 1b parameters, **from scratch**. a decoder-only language model, and none of what follows is about language.

shards

the corpus, in pieces



loader

reads what it can see



1b

quantisation-aware

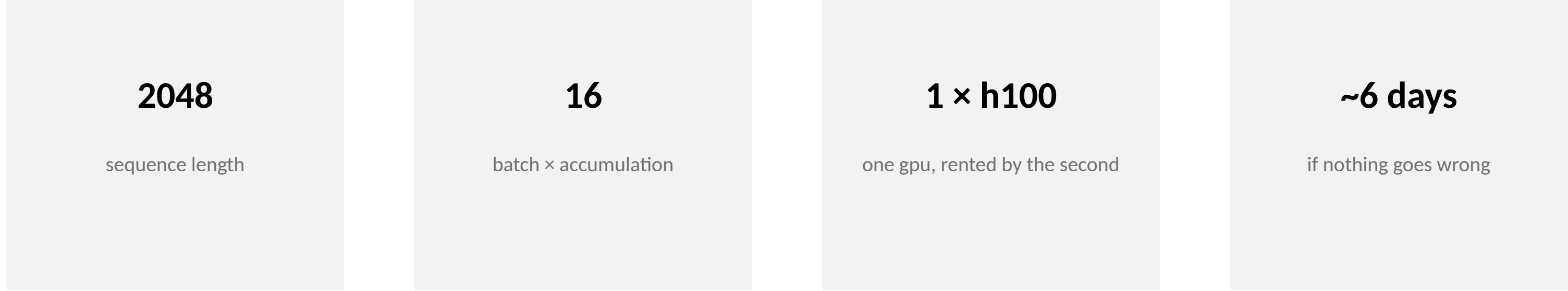


ckpt

back onto the volume

one loop. **the first box is the one that matters today.**

the corpus does not arrive all at once. **it arrives in boxes.** ten percent, twenty, thirty.



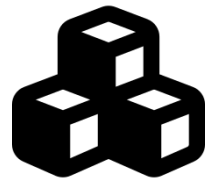
the run, in four numbers



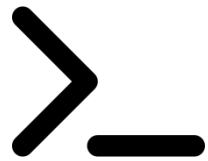
———— **pytorch 2.8.** cuda 12.8.



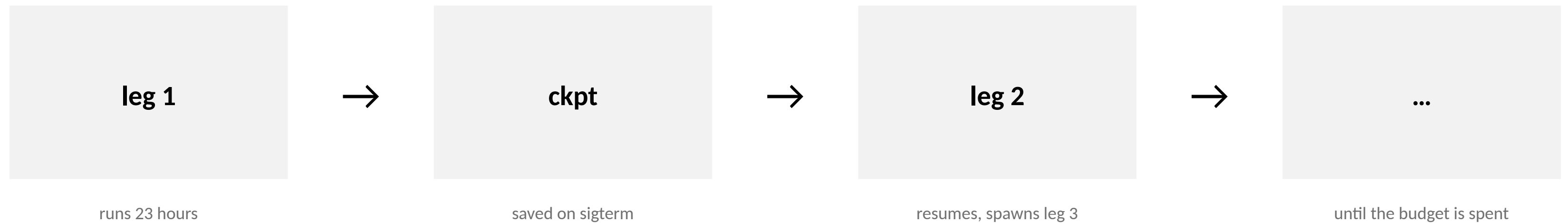
———— **modal.** h100 container, shared volume, a secret for the gated tokenizer.



———— **huggingface.** datasets and the tokenizer.



———— **one script.** checkpoints on sigterm, resumes from latest.



six days does not fit in a 24 hour container. **so it runs in legs.**

so here's the run that cost the most.

hour 1



prepare starts unpacking the corpus

hour 2



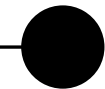
training starts. no errors anywhere

hour 6



the loss is climbing. **normal, early on.** i leave it

hour 24



it still has not come down. **not normal**

one day, and the only thing that went wrong **never raised anything**

0.95

train loss

vs.

5.6

validation loss

so i started guessing. in order.

guess one

~~the eval split is noisy~~

noise does not climb in a straight line for eighteen hours

guess two

~~it is overfitting~~

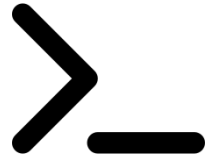
a 1b model does not overfit a corpus this size in a day

guess three

count the tokens it actually saw

took thirty seconds. should have been first

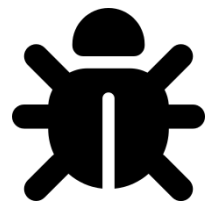
two hypotheses, then **arithmetic**



read the container logs by hand



pasted them into a chat window and argued with it



no dashboard, no trace, **no alert.** nothing was watching for this

and then i did the arithmetic.

steps

×

2048

×

16

=

96m

from the log

sequence

batch × accum

tokens. that is all it saw

four numbers already in the log. **i had never multiplied them.**

10%

vs.

90%

of the corpus it trained on

it never saw at all

the first shard, thirty times over. **that is not pretraining, that is memorisation.**

which is why it looked **brilliant on training data** and useless on anything real.

so why did it only ever see the first box?



a **cache**. nobody called it one.

prepare

still unpacking boxes



volume

one box visible, so far



train ×2

looked once, never again

and **both** trainers were fighting over the same box

a container only sees what another container wrote **after it asks again.**

i had never told the trainer **the data had finished loading.**

reload

ask the volume again

+

assert

refuse to start on a partial corpus

+

chain

survive the 24h ceiling

the whole fix is **three lines and a guard clause**

48

h100 hours. two containers, one full day each.

and none of this is special to me.



the foreground launch. my laptop's dns killed four remote jobs.



a twenty four hour cap. the run needed six days.

that cap is also **the only reason this cost 48 hours** and not a fortnight of credits.



two volumes, one name. ten very bad minutes.



a zero sized volume. the models expired like they never existed.

every one of these is **somebody's postmortem this quarter**. they are not exotic. they are the default.



—— **shared state.** who guarantees the input is complete before a consumer reads it?



—— **lifetime.** what keeps the remote work alive, and for how long?



—— **ownership.** whose job was it to notice?

hence, four checks. every run, no exceptions.



———— **smoke test.** one tiny pass end to end, on a cpu, before anything paid starts.



———— **hash the data.** if the hash matches, all of it loaded. if it does not, do not start.



———— **checkpoint.** every couple of hours, and push a copy to the hub.



———— **list the tiers.** take the cheaper gpu. the same mistake on it is a cheaper mistake.

2 minutes

what the smoke test costs

vs.

6 days

what it is deciding the shape of

the consumer asserts. **it does not trust.**

now. back to the question i started with.

91.3



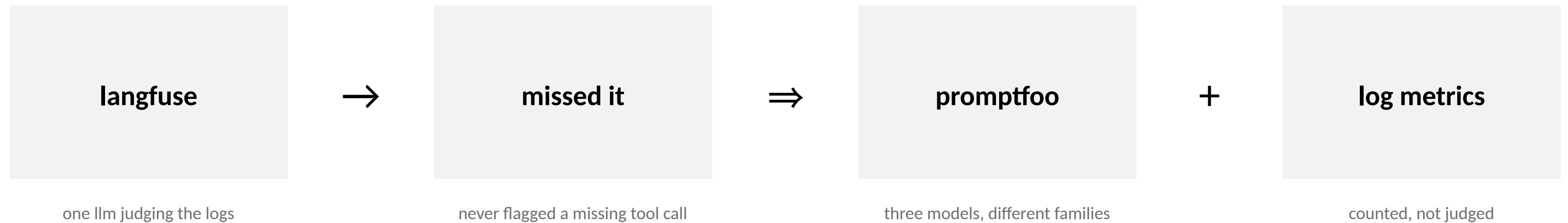
90.2

bf16, untouched

eight bit, no retraining, no calibration set

eight bits is free. six bits, at the small end, **is not a model any more.**

but how would you even know?



a judge grades the text it was given. **silence reads as a perfectly reasonable answer.**



three judges gave it a ten. it had stopped answering.

1.2%

of prompts it still answered, while scoring a perfect 100 on one category

every capability metric needs a liveness metric.

who signs off when the model changes numerically but not by version?

what evidence does your team require?

has anyone ever rejected a change on it?

so who signs it off at your place?